

FLUTTER STUDY GUIDE

CupertinoApp

Isang malalim na gabay sa pag-build ng iOS-style na apps gamit ang Flutter's Cupertino design language — puno ng paliwanag, halimbawa, at mga tips.

Root Widget

iOS Human Interface Guidelines

CupertinoThemeData

Navigation & Routing

Widget Catalog

Best Practices

Para sa mga Flutter developer na gustong matuto ng Cupertino design system

Table of Contents

Ito ang mga topics na tatalakayin natin — kumpleto, walang mamimiss.

01 Ano ang CupertinoApp?

02 CupertinoApp vs. MaterialApp

03 Mahahalagang Properties ng CupertinoApp

04 CupertinoThemeData — Pag-customize ng Theme

05 Pagbuo ng Screen: CupertinoPageScaffold at CupertinoNavigationBar

06 Multi-Tab na Apps: CupertinoTabScaffold

07 Navigation gamit ang CupertinoPageRoute

08 Localization sa CupertinoApp

09 Ang Cupertino Widget Catalog

10 Platform-Adaptive Design

11 Best Practices — Do's and Don'ts

12 Kumpletong Halimbawa ng App

13 Buod / Cheat Sheet

14 Practice Questions

1 Ano ang CupertinoApp?

Ang starting point ng bawat iOS-style Flutter app

Ang **CupertinoApp** ay isang *convenience widget* sa Flutter na nagbabalot (wraps) ng mga widgets na karaniwang kailangan para sa isang app na sumusunod sa **Apple's iOS Human Interface Guidelines**. Sa madaling salita, ito ang "root widget" o pinakaunang widget na ginagamit mo kapag gusto mong maging katulad ng native iOS app ang itsura at kilos ng iyong Flutter app.

Kung pamilyar ka na sa **MaterialApp** (na ginagamit para sa Android/Material Design apps), ang **CupertinoApp** ang katumbas nito para sa iOS design language. Nagbibigay ito ng:

- **Navigation stack** — pinapamahalaan kung paano naglilipat-lipat ng screens ang app mo
- **Theming** — global na kulay, brightness, at font styles sa buong app
- **Localization** — suporta sa iba't ibang wika at rehiyon
- **iOS-style widgets** — buttons, switches, dialogs na kamukha ng totoong iOS components

Tandaan

Ang **CupertinoApp** ay galing sa `package:flutter/cupertino.dart` — kailangan mo itong i-import bago mo magamit.

► Simpleng Halimbawa

Narito ang pinaka-basic na setup ng isang CupertinoApp. Ito na ang "Hello World" para sa Cupertino:

DART

```
import 'package:flutter/cupertino.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // Ito ang mahalagang bahagi:
  @override
  Widget build(BuildContext context) {
    return CupertinoApp(
      title: 'Aking Unang iOS App',
      theme: const CupertinoThemeData(
        primaryColor: CupertinoColors.systemBlue,
      ),
      home: HomeScreen(),
    );
  }
}
```

```
}  
}
```

Paliwanag linya-por-linya:

- `runApp()` — ito ang nagpapatakbo ng buong Flutter app, ipinapasa dito ang root widget
- `CupertinoApp(...)` — ito mismo ang gumagawa ng "app shell" na may navigation, theming, atbp.
- `title` — ginagamit ng operating system (halimbawa sa task switcher)
- `theme` — dito mo ise-set ang mga kulay at estilo
- `home` — ang unang screen na makikita ng user pagbukas ng app

Tip

Hindi mo kailangang gamitin ang `CupertinoApp` kung ang app mo ay para lang sa Android. Pero kung layunin mong magmukhang "native" sa iOS devices (o gumagawa ka ng platform-adaptive app), sulit itong pag-aralan.

2 CupertinoApp vs. MaterialApp

Dalawang design languages, iisang framework

Mahalagang maintindihan ang pagkakaiba ng dalawa dahil pareho silang "root widget" pero magkaiba ang *design philosophy*. Ang **Material Design** ay galing sa Google (karaniwan sa Android), habang ang **Cupertino** ay batay sa Apple's Human Interface Guidelines (iOS/iPadOS/macOS).

Aspeto	CupertinoApp	MaterialApp
Design system	Apple Human Interface Guidelines	Google Material Design
Theme class	CupertinoThemeData	ThemeData
Page layout widget	CupertinoPageScaffold	Scaffold
Top navigation bar	CupertinoNavigationBar	AppBar
Page transition	Slide-from-right (iOS style)	Fade-through / Material motion
Tab navigation	CupertinoTabScaffold	BottomNavigationBar
Buttons	CupertinoButton	ElevatedButton / TextButton
Ripple/press effect	Fade opacity lang, walang ripple	May "ripple" na animation
Karaniwang gamit	iOS-exclusive o platform-adaptive apps	Android-first o cross-platform apps

Mahalagang Paalala

Hindi ibig sabihin na kailangan mong pumili ng isa lang. Puwede mong pagsamahin ang dalawa — halimbawa, gamitin ang **MaterialApp** bilang pangunahing shell pero gamitin ang mga Cupertino widgets sa loob nito para sa mga iOS-specific na bahagi. Tatalakayin natin ito nang mas detalyado sa Section 10.

3 Mahahalagang Properties

Ang mga constructor parameters na dapat mong malaman

Tulad ng `MaterialApp`, ang `CupertinoApp` ay may maraming configurable na properties. Hindi lahat ay required — karamihan ay optional at may sensible defaults. Narito ang pinaka-importante:

Property	Type	Paliwanag
<code>home</code>	Widget	Ang default na screen na ipapakita kapag binuksan ang app
<code>theme</code>	CupertinoThemeData	Kontrolado nito ang kulay, brightness, at font ng buong app
<code>title</code>	String	Pangalan ng app na ginagamit ng OS (task switcher, atbp.)
<code>routes</code>	Map<String, WidgetBuilder>	Named-route table — mapa ng pangalan ng route papunta sa widget builder
<code>initialRoute</code>	String	Ang unang route na ipapakita, kapag gumagamit ka ng named routes
<code>onGenerateRoute</code>	Function	Dynamic na paggawa ng route — kapaki-pakinabang kung may arguments na kailangan ipasa
<code>navigatorKey</code>	GlobalKey<NavigatorState>	Nagbibigay-daan mag-navigate mula sa labas ng widget tree (e.g. sa isang service class)
<code>localizationsDelegates</code>	List	Nag-e-enable ng localized na strings at date/number formats para sa Cupertino widgets
<code>supportedLocales</code>	List<Locale>	Listahan ng mga locale (wika/rehiyon) na suportado ng app
<code>builder</code>	Function	Binabalot ang buong app — magandang gamitin para sa global overlays o custom theming logic
<code>debugShowCheckedModeBanner</code>	bool	Kung ipapakita ang "DEBUG" banner sa upper-right (default: true)
<code>showPerformanceOverlay</code>	bool	Nagpapakita ng performance graph overlay — para sa debugging

▶ Gamit ang Named Routes

Sa halip na direktang i-set ang `home`, puwede mo ring gamitin ang `routes` para sa mas structured na navigation, lalo na kung malaki ang app mo:

DART

```
CupertinoApp(  
  title: 'Multi-Screen App',  
  initialRoute: '/',  
  routes: {  
    '/': (context) => HomeScreen(),  
    '/profile': (context) => ProfileScreen(),  
    '/settings': (context) => SettingsScreen(),  
  },  
)
```

Pagkatapos, sa loob ng app, puwede kang mag-navigate gamit ang:

DART

```
Navigator.pushNamed(context, '/profile');
```

4 CupertinoThemeData

Kontrolin ang visual identity ng iyong iOS-style app

Ang `CupertinoThemeData` ang nagde-define ng "look and feel" ng app mo — kulay, brightness (light/dark mode), at typography. Pinapasa ito sa `theme` property ng `CupertinoApp`.

Field	Paliwanag
<code>brightness</code>	<code>Brightness.light</code> o <code>Brightness.dark</code> — kontrolado nito kung light mode o dark mode ang app
<code>primaryColor</code>	Main accent color — ginagamit sa mga buttons at active elements
<code>primaryContrastingColor</code>	Kulay na kontra sa <code>primaryColor</code> — para sa contrast (hal. text sa ibabaw ng button)
<code>barBackgroundColor</code>	Background color ng navigation bars at tab bars
<code>scaffoldBackgroundColor</code>	Background color sa likod ng buong <code>CupertinoPageScaffold</code>
<code>textTheme</code>	Isang <code>CupertinoTextThemeData</code> — kontrolado ang font styles (navTitleTextStyle, actionTextStyle, atbp.)

▶ Halimbawa: Custom Theme

DART

```
CupertinoApp(  
  theme: const CupertinoThemeData(  
    brightness: Brightness.dark,  
    primaryColor: CupertinoColors.activeOrange,  
    scaffoldBackgroundColor: CupertinoColors.black,  
    textTheme: CupertinoTextThemeData(  
      navTitleTextStyle: TextStyle(  
        fontSize: 20,  
        fontWeight: FontWeight.bold,  
        color: CupertinoColors.white,  
      ),  
    ),  
  ),  
  home: HomeScreen(),  
)
```


▶ Pag-access sa Theme sa Loob ng Widget

Sa loob ng anumang widget na nasa ilalim ng `CupertinoApp`, puwede mong kunin ang kasalukuyang theme gamit ang:

DART

```
final theme = CupertinoTheme.of(context);  
final color = theme.primaryColor;
```

Tip

Iwasan ang paggamit ng hardcoded na kulay tulad ng `Color(0xFF007AFF)`. Gamitin ang `CupertinoTheme.of(context)` para automatic na mag-adjust ang mga kulay kapag nag-switch ng light/dark mode.

5 Pagbuo ng Screen

CupertinoPageScaffold at CupertinoNavigationBar

Ang `CupertinoPageScaffold` ang standard layout structure para sa isang iOS-style na screen — ito ang Cupertino version ng `Scaffold` sa Material. Binubuo ito ng dalawang pangunahing bahagi: ang *navigation bar* sa itaas, at ang *child* na siyang main content.

Property	Paliwanag
<code>navigationBar</code>	Isang <code>CupertinoNavigationBar</code> na naka-pin sa itaas ng screen
<code>child</code>	Ang pangunahing content — kadalasan scrollable o interactive
<code>backgroundColor</code>	Nag-o-override sa background color ng scaffold
<code>resizeToAvoidBottomInset</code>	Kung ia-adjust ang layout kapag lumitaw ang keyboard

▶ Halimbawa: Basic na Screen

DART

```
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return CupertinoPageScaffold(
      navigationBar: const CupertinoNavigationBar(
        middle: Text('Inbox'),
        trailing: Icon(CupertinoIcons.add),
      ),
      child: SafeArea(
        child: ListView.builder(
          itemCount: 10,
          itemBuilder: (context, index) => CupertinoListTile(
            title: Text('Message ${index + 1}'),
          ),
        ),
      ),
    );
  }
}
```

▶ CupertinoNavigationBar — Mga Bahagi

- `leading` — widget sa kaliwang bahagi (kadalasan "Back" button)
- `middle` — karaniwan ay ang title/pangalan ng screen
- `trailing` — widget sa kanang bahagi (hal. add button, menu)

- `backgroundColor` — kulay ng nav bar background
- `previousPageTitle` — override sa text ng "Back" button

Tandaan

Awtomatikong lumalabas ang "Back" button sa `leading` kapag may naka-push na bagong page — hindi mo na ito kailangang manual na gawin, kaibahan ito sa AppBar ng Material.

6 Multi-Tab na Apps

CupertinoTabScaffold, CupertinoTabBar, at CupertinoTabView

Karamihan sa mga iOS apps (Instagram, App Store, Settings) ay gumagamit ng **bottom tabs** para sa main navigation. Ang **CupertinoTabScaffold** ang gumagawa nito — pinapanatili nitong laging visible ang tab bar habang ang bawat tab ay may sariling independent navigation stack.

Property	Paliwanag
<code>tabBar</code>	Isang <code>CupertinoTabBar</code> na may icons at labels para sa bawat tab
<code>tabBuilder</code>	Function na gumagawa ng content — kadalasan naglalagay ng <code>CupertinoTabView</code> dito
<code>controller</code>	Optional na <code>CupertinoTabController</code> para kontrolin ang active tab index programmatically

▶ Halimbawa: 3-Tab na App

DART

```
CupertinoTabScaffold(  
  tabBar: CupertinoTabBar(  
    items: const [  
      BottomNavigationBarItem(  
        icon: Icon(CupertinoIcons.home),  
        label: 'Home',  
      ),  
      BottomNavigationBarItem(  
        icon: Icon(CupertinoIcons.search),  
        label: 'Search',  
      ),  
      BottomNavigationBarItem(  
        icon: Icon(CupertinoIcons.settings),  
        label: 'Settings',  
      ),  
    ],  
  ),  
  tabBuilder: (context, index) {  
    switch (index) {  
      case 0:  
        return CupertinoTabView(builder: (_) => HomeScreen());  
      case 1:  
        return CupertinoTabView(builder: (_) => SearchScreen());  
      default:  
        return CupertinoTabView(builder: (_) => SettingsScreen());  
    }  
  }  
)
```

```
},  
)
```

Bakit CupertinoTabView?

Binabalutan ng **CupertinoTabView** ang bawat tab ng sarili nitong **Navigator**. Ibig sabihin, kung mag-push ka ng bagong page sa Tab 1, mananatiling naka-stay sa una nitong screen ang Tab 2 kahit mag-switch ka pabalik-balik. Ganito rin ang gawi ng totoong iOS apps.

7 CupertinoPageRoute

Native iOS na slide transition, walang extra code

Kapag gusto mong mag-navigate papunta sa bagong screen habang nasa loob ng isang CupertinoApp, gamitin ang `CupertinoPageRoute` sa halip na plain `MaterialPageRoute`. Awtomatiko nitong ilalapat ang `slide-from-right` na animation — ang klasikong iOS transition.

▶ Push papunta sa Bagong Screen

DART

```
Navigator.push(  
  context,  
  CupertinoPageRoute(  
    builder: (context) => const DetailPage(),  
  ),  
);
```

▶ Pag-pass ng Data papunta sa Bagong Screen

DART

```
Navigator.push(  
  context,  
  CupertinoPageRoute(  
    builder: (context) => DetailPage(itemId: 42),  
  ),  
);  
  
// Pagbabalik gamit ang result:  
final result = await Navigator.push(context, CupertinoPageRoute(  
  builder: (context) => SelectionPage(),  
));  
print('Napili: $result');
```

▶ Bumalik sa Nakaraang Screen

DART

```
Navigator.pop(context);  
  
// O magpasa ng result pabalik:  
Navigator.pop(context, 'napiling value');
```

Tandaan

Ang **CupertinoPageRoute** ay may built-in na "swipe from left edge to go back" gesture — libre mo na itong makukuha, walang dagdag na code.

8 Localization sa CupertinoApp

Suporta sa iba't ibang wika at rehiyon

Kung may mga user ka mula sa iba't ibang bansa, mahalagang i-configure ang localization. Ang mga Cupertino widgets (tulad ng date picker) ay may sariling mga built-in na strings na kailangan ng tamang delegate para lumitaw nang tama.

DART

```
import 'package:flutter_localizations/flutter_localizations.dart';

CupertinoApp(
  localizationsDelegates: const [
    DefaultCupertinoLocalizations.delegate,
    GlobalCupertinoLocalizations.delegate,
    GlobalWidgetsLocalizations.delegate,
  ],
  supportedLocales: const [
    Locale('en', 'US'),
    Locale('fil', 'PH'),
  ],
  home: HomeScreen(),
)
```

Karaniwang Error

Kung nakalimutan mo itong i-set at gumamit ka ng widget tulad ng `CupertinoDatePicker`, puwede kang makakuha ng error tungkol sa missing localization delegate sa console.

9 Ang Cupertino Widget Catalog

Mga pangunahing widgets na ginagamit sa loob ng CupertinoApp

Ang **CupertinoApp** mismo ay isang container lamang — ang totoong kapangyarihan nito ay nasa mga widgets na puwede mong gamitin sa loob nito. Narito ang pinaka-ginagamit:

CupertinoButton

Tappable button na may fade effect kapag pinindot (walang ripple)

CupertinoSwitch

iOS-style na on/off toggle switch

CupertinoTextField

Text input na may itsura ng iOS forms

CupertinoAlertDialog

Native-looking na alert/confirmation dialog

CupertinoActionSheet

Bottom sheet na may listahan ng actions/options

CupertinoPicker

Spinning wheel picker — para sa dates, values, o listahan

CupertinoDatePicker

Specialized picker para sa petsa at oras

CupertinoSlider

iOS-style na slider control

CupertinoActivityIndicator

Ang paikot-ikot na iOS loading spinner

CupertinoSegmentedControl

Segmented tab-like selector (hal. "Day / Week / Month")

CupertinoListTile

Row item na ginagamit sa mga settings-style na lists

CupertinoSearchTextField

Search bar na may itsura ng iOS search field

▶ Halimbawa: Alert Dialog

DART

```
showCupertinoDialog(  
  context: context,  
  builder: (context) => CupertinoAlertDialog(  
    title: const Text('I-delete ba?'),  
    content: const Text('Hindi na ito puwedeng i-undo.'),  
    actions: [  
      CupertinoDialogAction(  
        child: const Text('Cancel'),  
        onPressed: () => Navigator.pop(context),  
      ),  
    ],  
  ),  
);
```

```

    CupertinoDialogAction(
      isDestructiveAction: true,
      child: const Text('I-delete'),
      onPressed: () => Navigator.pop(context),
    ),
  ],
),
);

```

▶ Halimbawa: Button at Switch

DART

```

CupertinoButton.filled(
  child: const Text('I-save'),
  onPressed: () {
    // gawin ang aksyon dito
  },
)

CupertinoSwitch(
  value: isEnabled,
  onChanged: (bool value) {
    setState(() => isEnabled = value);
  },
)

```

10 Platform-Adaptive Design

Hindi mo kailangang pumili ng Cupertino LANG o Material LANG

Isa sa magandang bagay sa Flutter, puwede mong pagsamahin ang dalawang design system para "native" ang pakiramdam sa lahat ng platform. Narito ang mga common approach:

Approach	Paliwanag
Platform.isIOS check	Manual na pag-branch ng UI code sa pagitan ng Cupertino at Material widgets gamit ang <code>if</code> statements
Adaptive widgets	Mga widget tulad ng <code>Switch.adaptive()</code> na awtomatikong nagre-render ng tamang style base sa platform
CupertinoApp sa loob ng MaterialApp	Mag-nest ng <code>CupertinoTheme</code> para sa mga isolated na iOS-styled na section lang
Dedicated design system	Full commitment sa Cupertino para sa isang purely iOS-exclusive na app

▶ Halimbawa: Adaptive Switch

DART

```
Switch.adaptive(  
  value: isEnabled,  
  onChanged: (value) => setState(() => isEnabled = value),  
)  
// Awtomatikong magiging CupertinoSwitch look kapag naka-run sa iOS,  
// at Material Switch naman kapag naka-run sa Android.
```

▶ Halimbawa: Manual Platform Check

DART

```
import 'dart:io' show Platform;  
  
Widget buildButton() {  
  if (Platform.isIOS) {  
    return CupertinoButton(  
      child: const Text('Submit'),  
      onPressed: () {},  
    );  
  } else {  
    return ElevatedButton(  
      child: const Text('Submit'),  
      onPressed: () {},  
    );  
  }  
}
```

```
    );  
  }  
}
```

11 Best Practices

Mga dapat gawin at iwasan

✓ Gawin

- Gamitin ang CupertinoApp kapag iOS-only o Apple-first ang target ng app
- I-pair ito ng CupertinoThemeData para consistent ang branding
- Gamitin ang CupertinoIcons para tugma ang iconography
- I-test sa totoong iOS device o simulator — mahalaga ang motion at fonts
- Gamitin ang CupertinoTheme.of(context) sa halip na hardcoded na kulay
- Panatilihin ang bawat tab na may sariling Navigator sa CupertinoTabView

✗ Iwasan

- Huwag pagsamahin nang basta-basta ang Material widgets sa loob ng purong Cupertino flow
- Huwag kalimutan ang localizationsDelegates kung multi-language ang app
- Huwag mag-hardcode ng kulay — gamitin ang theme
- Huwag palampasin ang accessibility (dynamic text size, VoiceOver labels)
- Huwag gamitin ang MaterialPageRoute sa loob ng CupertinoApp — mawawala ang iOS transition

Panghuling Tip

Kung hindi ka sigurado kung Cupertino o Material ang gagamitin, tanungin mo ang sarili: "Sino ang audience ko?" Kung karamihan sa users mo ay gumagamit ng iPhone, mas magiging maganda ang karanasan nila kung gagamit ka ng Cupertino widgets.

12 Kumpletong Halimbawa

Buong app na pinagsasama ang lahat ng natutunan natin

Narito ang isang kumpletong, gumagana na halimbawa ng isang simpleng iOS-style na app na may tabs, navigation, theme, at ilang widgets. Basahin at subukang i-trace kung paano magkakaugnay ang bawat bahagi:

DART — main.dart

```
import 'package:flutter/cupertino.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return CupertinoApp(
      title: 'Notes App',
      debugShowCheckedModeBanner: false,
      theme: const CupertinoThemeData(
        primaryColor: CupertinoColors.systemBlue,
      ),
      home: const MainTabs(),
    );
  }
}

class MainTabs extends StatelessWidget {
  const MainTabs({super.key});

  @override
  Widget build(BuildContext context) {
    return CupertinoTabScaffold(
      tabBar: CupertinoTabBar(
        items: const [
          BottomNavigationBarItem(
            icon: Icon(CupertinoIcons.list_bullet),
            label: 'Notes',
          ),
          BottomNavigationBarItem(
            icon: Icon(CupertinoIcons.settings),
            label: 'Settings',
          ),
        ],
      ),
      tabBuilder: (context, index) {
```

```

        return CupertinoTabView(
          builder: (context) {
            return index == 0 ? const NotesListScreen() : const
SettingsScreen();
          },
        );
      },
    );
  }
}

class NotesListScreen extends StatelessWidget {
  const NotesListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return CupertinoPageScaffold(
      navigationBar: const CupertinoNavigationBar(
        middle: Text('Mga Tala'),
      ),
      child: SafeArea(
        child: ListView.builder(
          itemCount: 5,
          itemBuilder: (context, index) {
            return CupertinoListTile(
              title: Text('Tala #${index + 1}'),
              onTap: () {
                Navigator.push(
                  context,
                  CupertinoPageRoute(
                    builder: (context) => NoteDetailScreen(index: index),
                  ),
                );
              },
            );
          },
        ),
      ),
    );
  }
}

class NoteDetailScreen extends StatelessWidget {
  final int index;
  const NoteDetailScreen({super.key, required this.index});

  @override
  Widget build(BuildContext context) {
    return CupertinoPageScaffold(
      navigationBar: CupertinoNavigationBar(
        middle: Text('Tala #${index + 1}'),

```

```

    ),
    child: Center(
      child: CupertinoButton.filled(
        child: const Text('Bumalik'),
        onPressed: () => Navigator.pop(context),
      ),
    ),
  );
}

class SettingsScreen extends StatefulWidget {
  const SettingsScreen({super.key});
  @override
  State<SettingsScreen> createState() => _SettingsScreenState();
}

class _SettingsScreenState extends State<SettingsScreen> {
  bool notificationsOn = true;

  @override
  Widget build(BuildContext context) {
    return CupertinoPageScaffold(
      navigationBar: const CupertinoNavigationBar(
        middle: Text('Settings'),
      ),
      child: SafeArea(
        child: CupertinoListTile(
          title: const Text('Notifications'),
          trailing: CupertinoSwitch(
            value: notificationsOn,
            onChanged: (val) => setState(() => notificationsOn = val),
          ),
        ),
      ),
    );
  }
}

```

Ano ang natutunan natin dito?

Ang halimbawang ito ay gumagamit ng **CupertinoTabScaffold** (2 tabs), **CupertinoPageScaffold** (bawat screen), **CupertinoNavigationBar** (title ng bawat screen), **CupertinoPageRoute** (navigation papunta sa detail), at **CupertinoSwitch** (interactive na widget) — lahat ng natutunan natin, magkakasama sa isang tunay na app structure.

13 Buod / Cheat Sheet

Mabilisang pagbalik-tanaw sa lahat ng tinalakay

1

Root Widget

CupertinoApp ang bumubuo ng iOS-styled na Flutter app, tulad ng ginagawa ng MaterialApp para sa Material Design.

2

Theming

CupertinoThemeData ang nagko-control ng kulay, brightness, at typography ng buong app.

3

Structure

CupertinoPageScaffold, CupertinoNavigationBar, at CupertinoTabScaffold ang bumubuo ng screens.

4

Motion

CupertinoPageRoute ang nagbibigay ng awtomatikong iOS slide transition.

5

Widgets

Buttons, switches, pickers, dialogs — kumpletong native-style component library.

6

Adaptive

Puwedeng pagsamahin sa Material widgets para maging native ang app sa lahat ng platform.

▶ Quick Reference Table

Kailangan mo...	Gamitin ito
Root ng app	CupertinoApp
Screen layout	CupertinoPageScaffold
Top bar	CupertinoNavigationBar
Bottom tabs	CupertinoTabScaffold + CupertinoTabBar
Push new screen	Navigator.push() + CupertinoPageRoute
Alert/confirmation	showCupertinoDialog() + CupertinoAlertDialog
Button	CupertinoButton
Toggle	CupertinoSwitch
Loading spinner	CupertinoActivityIndicator

14 Practice Questions

Subukan mong sagutin bago tumingin sa notes — gamitin ito para i-check kung talagang natutunan mo

Pag-review

1. Ano ang pagkakaiba ng `CupertinoApp` at `MaterialApp` sa usapin ng page transition?
2. Anong property ng `CupertinoApp` ang gagamitin mo kung gusto mong mag-navigate mula sa labas ng widget tree (hal. sa isang service class)?
3. Ano ang layunin ng `CupertinoTabView` sa loob ng `CupertinoTabScaffold` ?
4. Bakit hindi dapat mag-hardcode ng kulay tulad ng `Color(0xFF007AFF)` sa halip na gamitin ang theme?
5. Anong widget ang gagamitin mo para gumawa ng confirmation dialog bago mag-delete ng isang item?
6. Paano ka gagawa ng button na iisa ang display sa iOS pero magiging Material-style naman sa Android, nang walang manual `if` statement?

Susunod na Hakbang

Subukan mong buuin ang sarili mong maliit na app gamit ang mga natutunan dito — halimbawa, isang simpleng To-Do list na may tabs, navigation papunta sa detail screen, at `CupertinoSwitch` para sa "completed" status. Ang pinakamabisang paraan para matuto ay ang mismong pag-code.

— Katapusan ng Study Guide. Good luck sa pag-build ng iyong susunod na Flutter app!